TECH_STACK.md: Encrypted P2P File & Text Transfer

> **For the coding agent:** Read `PRD.md` first, then this file. This file is the source of truth for *how* to build. Where the two disagree, this file wins, and any deviation is listed in Section 12. Do not add dependencies, services, or features that are not listed here without asking.

1. Decisions at a Glance

Concern	Decision
Language	TypeScript everywhere (strict mode)
Repo layout	pnpm workspaces monorepo
Frontend	React + Vite, plain CSS modules (no UI framework in v1)
Signaling server	Node.js (current LTS) + `ws` WebSocket library, in-memory sessions
Shared code	`packages/protocol`: message types and validation (zod)
Transport	WebRTC `RTCDataChannel` (ordered, reliable)
NAT traversal	Public STUN in v1; coturn TURN in P1
Key exchange	Ephemeral ECDH P-256 via Web Crypto, keys bound to DTLS fingerprints
Payload encryption	AES-256-GCM via Web Crypto, per-chunk nonce

Pairing	QR (high-entropy secret in URL fragment) + short code (rendezvous) + SAS verification
QR generate / scan	`qrcode` / `qr-scanner`
Large file saving	File System Access API (Chromium), Blob fallback elsewhere
PWA	`vite-plugin-pwa`
Tests	Vitest (unit), Playwright (end-to-end with two browser contexts)
Lint / format	ESLint + Prettier
Hosting	Static host for `apps/web`, small Node host for `apps/signaling`

Why TypeScript everywhere: WebRTC and Web Crypto only exist in the browser, so the client must be JS/TS. Using the same language for the signaling server lets both sides share one protocol package, which removes a whole class of mismatch bugs. A Java backend would add a second language for no benefit here.

Why no UI framework or state library: the app has about three screens. Plain React state plus CSS modules keeps the dependency list short and the code easy to read for a portfolio.

2. Development Environment (Windows 11)

- Install Node.js current LTS, then `corepack enable` and use **pnpm**.
- Use PowerShell or Git Bash; every command in this repo must work in PowerShell.
- Test on real devices: run the web app with `vite --host` and open it from a phone on the same Wi-Fi. Note: camera access (QR scanning) and Web Crypto need a **secure context**, so use `localhost` on the same machine or an HTTPS tunnel (for example `cloudflared` or `ngrok`) for phone testing.

- Git from the first commit, with small commits per feature, because this repo is also a GitHub portfolio piece.

3. Repository Structure

```
p2p-transfer/
├─ PRD.md
├─ TECH_STACK.md
├─ README.md
├─ package.json                # workspace root scripts
├─ pnpm-workspace.yaml
├─ tsconfig.base.json
├─ .eslintrc.cjs / eslint.config.js
├─ .prettierrc
├─ apps/
│  ├─ web/                     # React + Vite client (PWA
│  │  ├─ index.html
│  │  ├─ vite.config.ts
│  │  └─ src/
│  │     ├─ main.tsx
│  │     ├─ App.tsx
│  │     ├─ screens/           # Home, Create, Join, Tran
│  │     ├─ components/        # QRDisplay, QRScanner, Co
│  │     ├─ lib/
│  │     │  ├─ signaling.ts    # WebSocket client
│  │     │  ├─ peer.ts         # RTCPeerConnection + data
│  │     │  ├─ crypto.ts       # ECDH, HKDF, AES-GCM, SAS
│  │     │  ├─ transfer.ts     # chunking, backpressure,
│  │     │  ├─ session.ts      # pairing state machine
│  │     │  └─ words.ts        # word list for short code
│  │     └─ styles/
│  └─ signaling/               # Node.js signaling server
│     └─ src/
│        ├─ index.ts           # HTTP + WebSocket bootstr
│        ├─ sessions.ts        # in-memory session store
│        ├─ rateLimit.ts       # token bucket per IP
│        └─ config.ts
```

```
├─ packages/
│  └─ protocol/                  # shared message types + z
│     └─ src/index.ts
└─ tests/
   └─ e2e/                       # Playwright specs
```

4. Frontend (`apps/web`)

- **React 18+ with Vite and TypeScript (strict).** Function components and hooks only.
- **State:** `useReducer` for the pairing and transfer state machines. No Redux, Zustand, or similar in v1.
- **Routing:** none needed; a single-page flow driven by session state (`home → create | join → connected`).
- **QR:** `qrcode` to render the QR to a canvas; `qr-scanner` to read from the camera. Both run entirely client side.
- **PWA:** `vite-plugin-pwa` for the manifest and service worker. Add the Web Share Target in P1.
- **File saving:**
  - Chromium: `showSaveFilePicker()` and write chunks to a `FileSystemWritableFileStream` as they arrive, so memory stays constant.
  - Other browsers (Firefox, Safari): collect chunks into a `Blob` and trigger a download. Show a warning above a size threshold (suggest 1 GB) that this path uses RAM.
- **File reading:** `File.stream()` or `Blob.slice()` to read in 16 KB chunks; never `readAsArrayBuffer` on a whole file.
- **Styling:** CSS modules, mobile first (design at 360 px width), `prefers-color-scheme` for dark mode.

5. Signaling Server (`apps/signaling`)

- **Node.js + `ws`**, one process, no database. Sessions live in a `Map` and are deleted on expiry, completion, or disconnect.
- **Responsibilities (and nothing else):** create a session, let exactly

one second client join, relay opaque signaling messages between the two, then forget the session.
- **It must never** see file bytes, filenames, or encryption keys, and must never log message payloads.
- **Limits:**
  - Session expires after 5 minutes if unpaired.
  - Maximum 2 clients per session; a session is single-use.
  - Per-IP token bucket on create and join attempts (for example 10 per minute).
  - Maximum WebSocket message size of 64 KB.
  - Validate `Origin` against an allow-list from environment config.
- **Config via environment variables:** `PORT`, `ALLOWED_ORIGINS`, `SESSION_TTL_MS`.
- **Health endpoint:** `GET /health` returns 200.

Signaling protocol (defined in `packages/protocol`, validated with zod on both ends)

Message	Direction	Purpose
`create`	client → server	Ask for a new session
`created { sessionId, code }`	server → client	Session ready; `code` is the short code
`join { code }`	client → server	Second device joins by code
`joined` / `peer-joined`	server → clients	Both sides are present; start WebRTC
`signal { kind: "offer" | "answer" | "ice", data }`	client ↔ server ↔ client	Relayed verbatim; server does not interpret `data`
`error { reason }`	server → client	Expired, full, rate limited, invalid

`close`	either	End the session

6. WebRTC and Transfer Protocol

- **Peer connection:** one `RTCPeerConnection`, one ordered reliable `RTCDataChannel` for control and data (a second channel for bulk data can come later if needed).
- **ICE servers:** configurable list; start with public STUN, add TURN credentials from environment config in P1.
- **Chunking:** 16 KB messages. Respect backpressure: set `bufferedAmountLowThreshold` (for example 1 MB) and pause sending while `bufferedAmount` is above a high-water mark; resume on the `bufferedamountlow` event.
- **Application message types over the data channel** (all encrypted after key exchange):
  - `file-offer { id, name, size, mime }` → receiver shows Accept / Decline
  - `file-accept { id }` / `file-decline { id }`
  - `file-chunk { id, index, bytes }`
  - `file-end { id, sha256 }` → receiver verifies hash
  - `file-cancel { id }`
  - `text { body }`
- **Filename safety:** strip path separators and control characters, cap length, never auto-open files.

7. Cryptography

Use only the browser's **Web Crypto API** (`crypto.subtle`) plus the platform CSPRNG (`crypto.getRandomValues`). **Do not write any custom cryptographic primitive and do not add a general crypto library in v1.**

1. **Pairing secret.** The QR encodes a URL like `https://<host>/#s=<base64url 128-bit random secret>&c=<code>`. The fragment (`#...`) is never sent to any server.

2. **Key exchange.** Each side generates an ephemeral ECDH P-256 key pair and sends its public key over the data channel.
3. **Binding.** Derive the session key with HKDF-SHA-256 from: the ECDH shared secret, the pairing secret (when paired by QR), and both DTLS certificate fingerprints taken from the SDP. This stops a malicious signaling server from silently inserting itself.
4. **Payload encryption.** AES-256-GCM per chunk. Nonce = 96 bits built from a random per-direction prefix plus a counter, never reused with the same key. Include the file id and chunk index as additional authenticated data.
5. **SAS verification.** Derive a short authentication string (for example three words from a 2048-word list) from the same transcript hash and display it on both devices. For code-based pairing, the app must require the user to confirm the words match before enabling transfers. For QR pairing it is shown but optional.
6. **Ephemeral only.** Keys exist in memory for the session. Nothing is written to `localStorage`, `IndexedDB`, cookies, or disk.
7. **Short code format:** a number 1 to 999 plus two words from the 2048-word list (about 32 bits), for example `7-kiwi-lamp`. Single-use and 5-minute expiry. Online guessing is limited by the server rate limit.

8. Testing and Quality

- **Unit tests (Vitest):** crypto helpers (encrypt/decrypt round trip, nonce uniqueness, tamper detection), chunker and reassembler, protocol schema validation, session expiry and rate limiting.
- **End-to-end (Playwright):** open two browser contexts, pair by code, send a text, send a 50 MB file, assert the SHA-256 matches. Run on Chromium first; add Firefox and WebKit later.
- **Manual device matrix:** phone ↔ laptop on same Wi-Fi, phone on mobile data ↔ laptop on Wi-Fi (exercises STUN/TURN), and one strict-network case.
- **CI:** GitHub Actions running lint, type-check, unit tests, and the Playwright suite on each push.
- **TypeScript:** `strict: true`, no `any` without a comment explaining why.

9. Deployment

Piece	Recommendation	Notes
`apps/web`	Any static host with HTTPS (Cloudflare Pages, Netlify, or Vercel)	HTTPS is mandatory for camera and Web Crypto.
`apps/signaling`	A small always-on Node host (Render, Fly.io, or Railway)	Must support WebSockets. Check each provider's current free-tier limits before choosing.
TURN (P1)	Self-hosted coturn on a small VM, or a hosted TURN service	Needed for strict NATs. Use short-lived credentials, never hard-code a static password in the client.

Set a strict **Content-Security-Policy** on the web app (no inline scripts, `connect-src` limited to the signaling host) and add `Referrer-Policy: no-referrer`.

10. Build Order for the Agent

Complete each phase and show it working before starting the next. Commit after each phase.

1. **Scaffold:** pnpm workspace, TypeScript configs, ESLint, Prettier, empty `web`, `signaling`, and `protocol` packages that build.
2. **Signaling server:** create/join/relay with expiry and rate limiting, plus unit tests.
3. **Connect two tabs:** pairing by code, WebRTC connected, exchange a text message in plaintext (proof of concept).
4. **File transfer:** chunking, backpressure, accept/decline, progress, cancel, hash verification. Test with a 1 GB file for constant memory use.
5. **Crypto layer:** ECDH + HKDF + AES-GCM, DTLS fingerprint

binding, SAS screen. All data now encrypted; tests for tampering.
6. **QR pairing:** generate and scan, fragment-carried secret.
7. **PWA and polish:** install prompt, error states, dark mode, accessibility pass.
8. **P1 items:** multi-file queue, TURN fallback, share target, resume.
9. **Docs:** README with architecture diagram, threat model, setup steps, and demo GIF.

11. Rules for the Agent

- Never log message payloads, filenames, codes, or keys on the server.
- Never send the URL fragment or any secret to a server.
- Never reuse an AES-GCM nonce with the same key.
- Do not store anything in browser storage.
- Prefer the platform APIs (Web Crypto, Streams, File System Access) over third-party packages.
- Every new dependency needs a one-line justification in the pull request or commit message.
- Keep functions small and typed; add a short comment explaining *why* for anything security-related.
- When something in this file is ambiguous or seems wrong, stop and ask instead of guessing.

12. Deviations from the PRD

- **PAKE is deferred to v1.1.** The PRD proposed a PAKE (CPace/SPAKE2) for the short code. There is no widely trusted, drop-in browser PAKE library, and hand-implementing one is risky. For v1 the short code is a rendezvous identifier only; authentication comes from the DTLS-bound key derivation plus **mandatory SAS confirmation** for code pairing, with rate-limited, single-use, 5-minute codes. Revisit with an audited CPace implementation after the MVP works.
- **Dark mode** is included in the polish phase instead of being P2.

13. First Prompt to Paste into Antigravity

```
Read PRD.md and TECH_STACK.md in this workspace fully bef
Then execute Phase 1 (Scaffold) from Section 10 of TECH_S
Show me the folder structure and the commands to run it o
and stop for my review before starting Phase 2.
```